

Examen 1

Nombre: _____

Fecha: _____

Grupo: _____

Calificación: _____ / 20

Instrucciones: selecciona la única respuesta correcta en cada pregunta. Cada pregunta vale 1 punto.

1. ¿Cuál describe mejor a Node.js?

- a) Un lenguaje de programación para servidores
- b) Un runtime de JavaScript que permite ejecutar código fuera del navegador
- c) Un framework web para construir APIs
- d) Un gestor de paquetes para JavaScript

2. ¿Cuál de las siguientes es responsabilidad del backend y no del frontend?

- a) Administrar el estado de la interfaz
- b) Capturar y validar inicialmente los datos de un formulario
- c) Autenticar y autorizar operaciones y aplicar reglas de negocio
- d) Presentar la información al usuario

3. En una API web, el "contrato" normalmente incluye:

- a) Endpoint, método HTTP, parámetros o body, headers, formato de respuesta y códigos de estado
- b) Únicamente la URL del servidor
- c) El código fuente completo del backend
- d) La estructura de la base de datos

4. ¿Qué método HTTP se usa para modificar parcialmente un recurso?

- a) GET
- b) POST
- c) PUT
- d) PATCH

5. ¿Cuál afirmación sobre el modelo *single-threaded* de Node.js es correcta?

- a) El proceso completo usa un solo hilo y no puede aprovechar varios núcleos
- b) El código JavaScript de la aplicación corre en un hilo principal asociado al event loop, y aun así se pueden usar `worker_threads`, `cluster` o varias réplicas
- c) Node.js crea un hilo del sistema operativo por cada conexión entrante
- d) Node.js no puede atender operaciones de I/O de forma concurrente

6. ¿Por qué una operación CPU-bound es problemática en Node.js?

- a) Porque Node.js no puede ejecutar cálculos matemáticos
- b) Porque obliga a crear un hilo nuevo por cada petición
- c) Porque bloquea el event loop y retrasa todas las demás solicitudes
- d) Porque impide que el servidor use el protocolo HTTP

7. ¿Qué problema resuelve NVM que NPM no resuelve?

- a) Instalar y cambiar entre distintas versiones de Node.js
- b) Instalar dependencias del proyecto
- c) Ejecutar scripts definidos en package.json
- d) Publicar paquetes en el registro

8. Sobre monolitos y microservicios, ¿cuál afirmación es correcta?

- a) Los microservicios siempre son una mejor arquitectura
- b) Un monolito no puede tener módulos separados internamente
- c) Los microservicios eliminan la complejidad operativa
- d) La elección depende del dominio, el equipo y las necesidades operativas

9. ¿Cuál de las siguientes asociaciones entre comando de NPM y su función es correcta?

- a) `npm init -y`: crea el package.json con valores por defecto, sin preguntas interactivas
- b) `npm install express`: instala Express de forma global en el sistema para todos los proyectos
- c) `npm install --save-dev nodemon`: registra nodemon como dependencia indispensable para ejecutar la app en producción
- d) `npm audit`: ejecuta el script test definido en package.json

10. Si package.json define los scripts start, dev y test, ¿cómo se ejecuta el script dev?

- a) `npm dev`
- b) `npm start dev`
- c) `npm run dev`
- d) `node dev`

11. ¿Cuál es la función del archivo package-lock.json?

- a) Declara qué librerías necesita el proyecto
- b) Contiene el código de las dependencias instaladas
- c) Define el punto de entrada del paquete
- d) Registra las versiones exactas de las dependencias instaladas

12. Para la petición GET /api/items?page=2, ¿cómo se accede al valor de page en Express?

- a) `request.query.page`
- b) `request.params.page`
- c) `request.body.page`
- d) `request.headers.page`

- 13. ¿Cuál es la relación correcta entre cookie y sesión?**
- a) La cookie almacena todos los datos de la sesión en el navegador
 - b) La cookie solo transporta un identificador; los datos de la sesión viven en el servidor
 - c) La sesión se guarda siempre en el navegador y la cookie en el servidor
 - d) Cookie y sesión son exactamente lo mismo
- 14. ¿Qué ocurre si un middleware no llama a `next()` ni envía una respuesta?**
- a) Express responde automáticamente con 200
 - b) Express salta al middleware de errores
 - c) Express responde con 404
 - d) La petición se queda esperando
- 15. Una ruta usa `requireSession` y luego `requireRole('admin')`. Si la petición llega sin sesión, ¿qué ocurre?**
- a) Responde 401 y no se ejecuta `requireRole` ni el handler final
 - b) Responde 403 tras ejecutar `requireRole`
 - c) Se ejecuta el handler final de todos modos
 - d) Responde 404 porque la ruta no coincide
- 16. ¿Cómo reconoce Express a un middleware de manejo de errores?**
- a) Por su nombre, que debe ser `errorHandler`
 - b) Porque se registra con `app.error()`
 - c) Por su firma de cuatro parámetros (`error`, `request`, `response`, `next`) y porque se define al final
 - d) Porque siempre devuelve el status 500
- 17. ¿Dónde debe colocarse el middleware que captura rutas inexistentes (route not found, 404)?**
- a) Antes de `express.json()`
 - b) Después de todas las rutas reales
 - c) Antes de todas las rutas
 - d) Dentro de cada controlador
- 18. Respecto a `app.use(express.static(path.join(__dirname, 'public')))`, ¿cuál afirmación es correcta?**
- a) `__dirname` es una carpeta que se debe crear manualmente
 - b) Se debe escribir un route handler por cada archivo dentro de `public/`
 - c) Concatenar `__dirname + '/public'` es más robusto que usar `path.join`
 - d) Sirve los archivos de `public/` directamente, y `path.join` arma la ruta de forma segura en cualquier sistema operativo

19. En GET /api/items/:id, ¿qué respuesta se devuelve si el id es 999 (un entero positivo válido) pero no existe ese item?

- a) 400
- b) 200 con null
- c) 404
- d) 500

20. En la estructura propuesta para el proyecto, ¿qué función cumplen routes/ y controllers/?

- a) routes/ define qué URL dispara qué función; controllers/ contiene la lógica que responde cada petición
- b) routes/ contiene la lógica de negocio; controllers/ define las URLs
- c) routes/ guarda los archivos estáticos; controllers/ guarda las dependencias
- d) Ambas carpetas contienen únicamente configuración de NPM